

Process Synchronization

When two or more processes work together, their order of execution must be properly maintained. Otherwise, conflicts can happen and the program may produce wrong results.

A **cooperative process** is a process that can affect another process or can be affected by another process.

Such processes need to be **synchronized** so that their execution happens in the correct order.

Producer-Consumer Problem

The **Producer-Consumer Problem** is a classical problem of process synchronization.

In this problem, there is:

- One **Producer**
- One **Consumer**
- One shared **memory buffer**

The Producer produces some items and puts them into the buffer.

The Consumer takes the items from the buffer and consumes them.

The buffer has a **fixed size** and is shared by both Producer and Consumer.

The Producer's job is: 1. Produce an item. 2. Put the item into the buffer. 3. Start producing the next item.	The Consumer's job is: 1. Take an item from the buffer. 2. Consume the item. 3. Take the next item.
--	--

What is the Problem?

The following problems can occur in the Producer-Consumer Problem:

- The Producer can put data into the buffer only when the buffer is **not full**.
- If the buffer is full, the Producer must wait.
- The Consumer can take data from the buffer only when the buffer is **not empty**.

- If the buffer is empty, the Consumer must wait.
- Producer and Consumer should not access the shared buffer at the same time.

Let's understand the code.

Producer Code

```
int count = 0;
void producer( void )
{
    int itemP;
    while(1)
    {
        Produce_item(item P)
        while( count == n);
        buffer[ in ] = item P;
        in = (in + 1)mod n
        count = count + 1;
    }
}
```

Load Rp, m[count]
Increment Rp
Store m[count], Rp

Consumer Code

```
int count;
void consumer(void)
{
    int itemC;
    while( 1 )
    {
        while( count == 0 );
        itemC = buffer[ out ];
        out = ( out + 1 ) mod n;
        count = count - 1;
    }
}
```

Load Rc, m[count]
Decrement Rc
Store m[count], Rc

Understanding Producer and Consumer Code

Producer Code Explanation

Suppose the Producer wants to produce an item "F".

The Producer enters the producer() function.

while(1) is always true, so the loop continues forever.

The Producer tries to insert:

itemP = F

Before inserting, it checks:

while(count == n);

If count == n, the buffer is full, so the Producer waits.

The semicolon after the while loop means that if the condition is true, the Producer stays in that loop.

If the condition is false, the Producer continues.

Suppose:

- Buffer size = 8
- count = 5
- in = 5

Then:

Buffer[in] = itemP;

becomes:

Buffer[5] = F;

So F is inserted at position 5.

Now:

in = (in + 1) mod n;

Therefore:

in = (5 + 1) mod 8

in = 6

So in = 6.

This means position 6 is now the next empty position.

After inserting F, the buffer contains 6 items.

Now: count = count + 1;

0	A
1	B
2	C
3	D
4	E
5	
6	
7	

0	A
1	B
2	C
3	D
4	E
5	F
6	
7	

This operation can be divided into three steps.

Step 1: Load

$R_p = \text{count};$

The current value of count is 5, so:

$R_p = 5$

Step 2: Increment

$R_p = R_p + 1;$

Now:

$R_p = 6$

Step 3: Store

$\text{count} = R_p;$

So count become 6.

Context Switch

Suppose a **context switch** happens after Step 2 and before Step 3.

The CPU now switches from the Producer to the Consumer.

The Producer has not yet stored $R_p = 6$ into count.

So the actual value of count is still:

$\text{count} = 5$

Now the Consumer starts executing.

Consumer Code Explanation

Suppose the Consumer wants to consume the first item, "A".

The Consumer enters the consumer() function.

Again:

while(1)

is always true.

The Consumer checks:

while(count == 0);

Since:

count = 5

the condition is false.

Therefore, the Consumer can continue.

The Consumer executes:

itemC = Buffer[out];

Since:

out = 0

the Consumer gets:

itemC = Buffer[0];

Suppose Buffer[0] = A.

Therefore:

itemC = A

0	
1	B
2	C
3	D
4	E
5	F
6	
7	

Now the Consumer moves out to the next position:

$\text{out} = (\text{out} + 1) \bmod n;$

Therefore:

$\text{out} = (0 + 1) \bmod 8$

$\text{out} = 1$

So:

$\text{out} = 1$

The item A has now been removed.

Updating Count

The Consumer performs:

$\text{count} = \text{count} - 1;$

This operation is divided into three steps.

Step 1: Load

The Consumer copies the current value of count into register Rc.

$\text{Rc} = \text{count};$

Therefore:

$\text{Rc} = 5$

Step 2: Decrement

$\text{Rc} = \text{Rc} - 1;$

Therefore:

$\text{Rc} = 4$

Step 3: Store

$\text{count} = \text{Rc};$

Therefore:

$\text{count} = 4$

Now the correct value of count is 4.

Another Context Switch

Now suppose a context switch happens again.

The CPU returns to the Producer.

The Producer continues from where it stopped.

Remember that the Producer had already incremented:

$\text{Rp} = 6$

but had not yet stored it into count.

Now it performs:

$\text{count} = \text{Rp};$

Therefore:

count = 6

But this is wrong.

Why?

Because:

- Producer added one item.
- Consumer removed one item.
- The buffer originally had 5 items.
- After Producer adds one item → 6.
- After Consumer removes one item → 5.

So the correct count should be:

count = 5

But because of the context switch, the Producer overwrites the Consumer's value of 4 with its old value 6.

Therefore, the final value becomes incorrect.

This situation is called a **Race Condition**.

This is the main problem in the Producer-Consumer Problem.

2)Race Condition

There are two types of processes.

1. Independent Process

Two processes are called **independent processes** when the execution of one process does not affect the execution of the other process.

Example:

If Process P1 works on its own data and Process P2 works on different data, P1 does not affect P2.

2. Cooperating Process

Two processes are called **cooperating processes** when the execution of one process affects the execution of another process.

This usually happens because the processes share something.

For example, they may share:

- Variables,Memory,Code,CPU,Printer,Scanner,Other resources

Cooperating processes must be synchronized to get the correct result.

Race Condition

A **Race Condition** occurs when two or more processes execute at the same time and access shared data without proper synchronization.

They may not execute in the correct order.

As a result, the final data may become incorrect.

Race conditions can cause:

- Data inconsistency, Data loss, Wrong results

3) Process Synchronization

Process Synchronization helps maintain the correctness of shared data.

It ensures that when multiple cooperating processes access shared data at the same time, they do not create inconsistent results.

For example:

P1	P2
{	{
Read(A);	Read(A);
A = A * 2;	A = A + 5;
Write(A);	Write(A);
}	}

Here, both processes are using the same variable A.

If they execute at the same time without synchronization, the final value of A may be wrong.

4) The Critical Section Problem

A **Critical Section** is the part of a program where a process accesses a shared resource.

A shared resource can be:

- Memory location
- Data structure
- CPU
- I/O device
- Shared variable
- File

Only one process should execute the critical section at a time when mutual exclusion is required.

The Operating System must control which process can enter the critical section and which process must wait.

The **Critical Section Problem** is used to create rules or protocols that prevent Race Conditions.

To synchronize cooperating processes, we need to solve the Critical Section Problem.

A good solution should satisfy the following conditions.

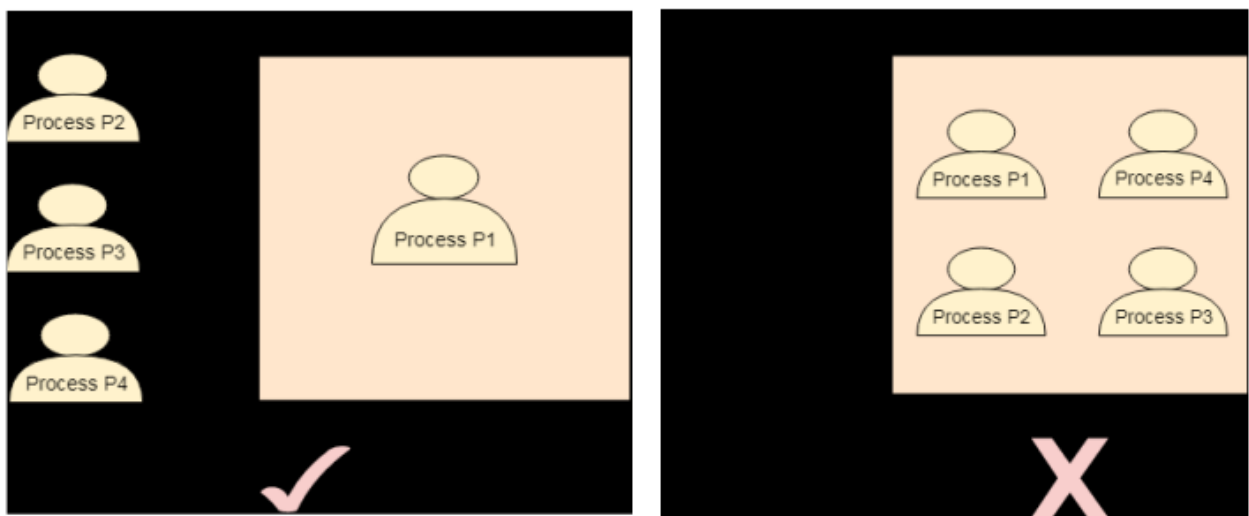
Requirements of Synchronization Mechanisms

There are two main categories:

- Primary requirements
- Secondary requirements

Primary Requirements

1. Mutual Exclusion



Mutual Exclusion means that if one process is executing inside the Critical Section, another process cannot enter the Critical Section at the same time.

In simple words:

Only one process can be inside the Critical Section at a time.

2. Progress

Progress means that if a process does not need to enter the Critical Section, it should not stop another process from entering it.

In simple words:

A process that does not need the Critical Section should not unnecessarily block other processes.

Secondary Requirements

1. Bounded Waiting

Bounded Waiting means that a process should not have to wait forever to enter the Critical Section.

There should be a limit on how long a process has to wait.

In simple words:

Every process should eventually get a chance to enter the Critical Section.

2. Architectural Neutrality

The synchronization mechanism should not depend on only one computer architecture.

If the solution works on one architecture, it should also work on other architectures.

5)Semaphore

A **Semaphore** is a synchronization tool used to solve the Critical Section Problem and avoid Race Conditions.

A semaphore S is an integer variable.

It can be accessed using two atomic operations:

1. wait()
2. signal()

These operations are **atomic**, which means they are performed as one complete operation.

When one process changes the semaphore value, another process cannot change the same semaphore value at exactly the same time.

Types of Semaphore

There are mainly two types:

1. **Counting Semaphore**
2. **Binary Semaphore / Mutex**

1. Counting Semaphore

A **Counting Semaphore** is used when more than one process is allowed to enter a critical section or access a limited number of resources at the same time.

For example, suppose there are 3 identical resources.

A counting semaphore can have a value of 3.

This means up to 3 processes can use the resources at the same time.

The basic structure is:

```
struct Semaphore
{
    int value;
    queue type L;
}
```

Here:

- value tells how many processes can access the resource.
- L is a queue containing blocked processes.

Down Operation Down (Semaphore S) { S.value = S.value - 1; if (S.value < 0) { put_process(PCB) in L; Sleep(); } else return; } When a process wants to enter: 1. The semaphore value is decreased by 1. 2. If the value becomes negative, the process is blocked. 3. The process is added to the waiting queue. If the value is not negative, the process can continue.	Up Operation Up (Semaphore S) { S.value = S.value + 1; if (S.value <= 0) { select a process from L; wake-up(); } } When a process leaves: 1. The semaphore value is increased by 1. 2. If there are waiting processes, one of them can be woken up. The value of the counting semaphore represents the number of available resources. A process first decreases the semaphore value by 1. If the value becomes negative, the process is placed in the blocked queue. Otherwise, it can enter the Critical Section.
--	--

2. Binary Semaphore or Mutex

A **Binary Semaphore** is used when only one process should enter the Critical Section at a time.

It has only two possible values: 0 1

Because it allows only one process at a time, it provides **Mutual Exclusion**.

A Binary Semaphore can be represented as:

Struct Bsemaphore

```
{  
    enum Value(0,1);  
    Queue type L;  
}
```

Here:

- 0 means the resource is not available.
- 1 means the resource is available.
- L contains blocked processes.

Down Operation

Down (Bsemaphore S)

```
{  
  if (S.value == 1)  
  {  
    S.value = 0;  
  }  
  else  
  {  
    put the process (PCB) in S.L;  
    sleep();  
  }  
}
```

If the value is 1, the process can enter.

The value is then changed to 0.

This prevents another process from entering at the same time.

If the value is already 0, the process is placed in the waiting queue and sleeps.

Up Operation

Up (Bsemaphore S)

```
{  
  if (S.L is empty)  
  {  
    S.Value = 1;  
  }  
  else  
  {  
    Select a process from S.L;  
    Wakeup();  
  }  
}
```

If there are no waiting processes, the semaphore becomes 1.

If there are waiting processes, one waiting process is selected and woken up.

6)Classic Problems of Synchronization

There are three famous synchronization problems:

1. **Bounded-Buffer Problem / Producer-Consumer Problem**
2. **Readers-Writers Problem**
3. **Dining-Philosophers Problem**

1)Bounded-Buffer Problem / Producer-Consumer Problem Using Semaphores

Semaphores can be used to solve the Producer-Consumer Problem.

The main advantages are:

- They prevent Race Conditions.

- They provide proper synchronization.
- They allow multiple items to be stored in the buffer.
- They prevent the Producer from adding to a full buffer.
- They prevent the Consumer from removing from an empty buffer.

Suppose the buffer contains n spaces.

Each space can store one item.

We use three semaphores:

1. mutex Used to provide Mutual Exclusion. Initial value: mutex = 1	2. empty Counts the number of empty spaces. Initial value: empty = n	3. full Counts the number of filled spaces. Initial value: full = 0
Producer Process Define n=100; Int buffer[n]; Csemaphore empty=n; Full=0; Bsemaphore mutex=1; Void produce(void) { Int itemp, in=0; Produce_item(itemp) While(true) { Down(empty); Down(mutex); // add item to buffer Buffer[in]=itemp; In=(in+1)mod n; Up(mutex); Up(full); } }		Consumer Process Void consumer(void) { Int itemc,out=0; While(true) { Down(full); Down(mutex); // remove item from buffer Itemc=buffer[out]; Out=(out+1)mod n; Up(mutex); Up(empty); // consume the item Process_item(itemc); } }

Explanation First: Down(empty); The Producer checks whether an empty space is available. Then: Down(mutex); The Producer gets exclusive access to the buffer. The item is inserted: Buffer[in] = itemp; Then in is moved to the next position: In = (in+1) mod n; After inserting the item: Up(mutex); The Producer releases the buffer. Then: Up(full); The number of full spaces is increased	Explanation First: Down(full); The Consumer checks whether at least one item is available. Then: Down(mutex); The Consumer gets exclusive access to the buffer. The item is removed: Itemc = buffer[out]; Then out moves to the next position: Out = (out+1) mod n; After removing the item: Up(mutex); The Consumer releases the buffer. Then: Up(empty); The number of empty spaces is increased. Finally: Process_item(itemc);
---	---

The Consumer consumes the item.

We can think of this process in two ways:

- The Producer produces **full buffers** for the Consumer.
- The Consumer produces **empty buffers** for the Producer.

2)Readers-Writers Problem

Many processes may share the same data object, such as:

- File
- Variable
- Database
- Memory

These processes can be divided into two groups:

1. **Readers**
2. **Writers**

Readers

Readers only want to read the shared data.

Multiple Readers can read the shared data at the same time.

There is normally no problem when multiple Readers read together.

Writers

Writers want to change or update the shared data.

A Writer may read and write the data.

A problem occurs if a Writer and another process access the shared data at the same time.

For example:

- Writer + Reader at the same time
- Writer + Writer at the same time

This can make the data inconsistent.

Therefore:

When a Writer is using the shared object, no Reader or other Writer should use it.

This is called the **Readers-Writers Problem**.

There are different versions of this problem.

First Readers-Writers Problem

The first Readers-Writers Problem says:

A Reader should not wait unnecessarily just because another Reader is reading.

If no Writer is using the shared object, multiple Readers can read at the same time.

However, if a Writer is already using the object, Readers must wait.

In this solution, Writers may suffer from **starvation**.

That means a Writer may have to wait for a very long time.

Second Readers-Writers Problem

The second Readers-Writers Problem gives priority to waiting Writers.

If a Writer is waiting to access the shared object, new Readers should not start reading.

The Writer should get a chance to write as soon as possible.

In this solution, Readers may suffer from **starvation**.

So:

- First problem → Writers may starve.
- Second problem → Readers may starve.

Solution for First Readers-Writers Problem

The Reader and Writer processes use:

Semaphore mutex, DB or wrt;

int read_count;

Initial values:

mutex = 1

DB = 1

read_count = 0

Purpose of mutex

mutex protects the read_count variable.

Only one process can change read_count at a time.

Purpose of read_count

read_count tells us how many Readers are currently reading the shared object.

Purpose of DB

DB controls access to the shared database or shared object.

It provides Mutual Exclusion for Writers.

The first Reader locks DB.

Other Readers can then read at the same time.

When the last Reader finishes, it releases DB.

If a Writer is currently in the Critical Section and many Readers are waiting:

- One Reader may wait on DB.
- Other Readers may wait on mutex.

When the Writer executes:

signal(DB)

either a waiting Reader or a waiting Writer may continue, depending on the scheduling mechanism.

Writer Process

Semaphore mutex=1;

Db=1;

Read_count=0;

Void writer(void)

While(true)

{

Down(DB);

// writing is performed

Up(DB);

}

The Writer:

1. Performs Down(DB).
2. Gets access to the shared object.
3. Performs writing.
4. Performs Up(DB).

Releases the shared object.

Reader Process

Void Reader(void)

While(true)

{

Down(mutex);

read_count++;

if (read_count == 1)

Down(DB);

Up(mutex);

// reading is performed

Down(mutex);

read_count--;

if (read_count == 0)

Up(DB);

Up(mutex);

}

Simple Explanation

When a Reader enters:

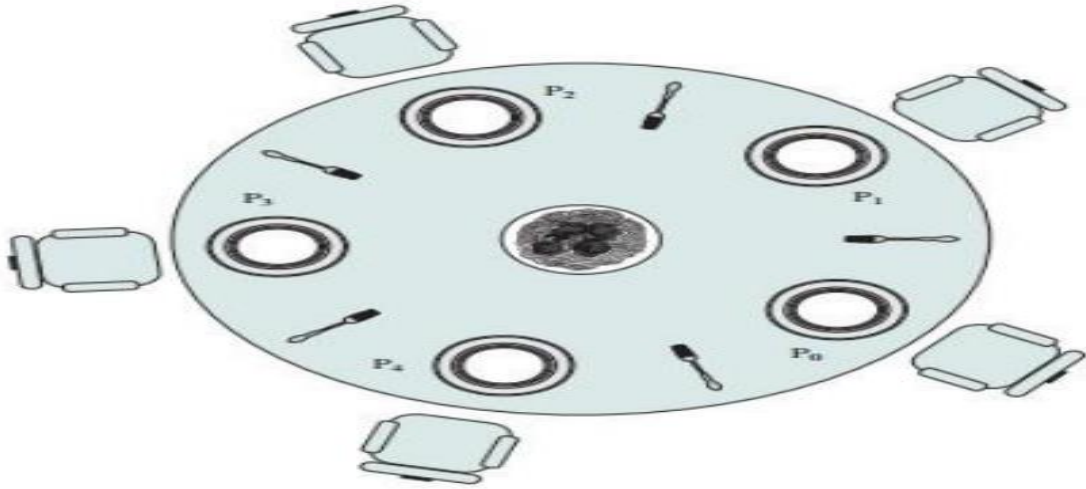
1. It locks mutex.
2. It increases read_count.
3. If it is the first Reader, it locks DB.
4. It releases mutex.
5. It performs reading.

When a Reader finishes:

1. It locks mutex.
2. It decreases read_count.
3. If it is the last Reader, it releases DB.
4. It releases mutex.

This allows multiple Readers to read together but prevents Writers from changing the data while Readers are using it.

3) Dining-Philosophers Problem



- ❖ **5 friends (philosophers)** sit around a round dinner table.
- ❖ They only do two things: **Think** and **Eat**.
- ❖ There are **5 chopsticks (or spoons)** on the table. One chopstick is placed between each friend.
- ❖ To eat, a friend **must pick up 2 chopsticks**—the one on their left and the one on their right.
- ❖ When they finish eating, they put both chopsticks down and start thinking again.

❖ **The Problem: Deadlock (Stuck Forever)**

Imagine all 5 friends sit down at the exact same time and follow this simple rule:
"Pick up your left chopstick first, then pick up your right chopstick."

- Every friend will pick up their **left chopstick** at the same moment.
- Now, all 5 chopsticks are taken.
- Next, every friend tries to grab their **right chopstick**, but it is already taken by the person next to them.
- No one can eat, and **no one will drop their chopstick**

until they eat.

- They will **wait forever and starve**. In computers, this frozen state is called a **Deadlock**.

❖ **Easy Fixes (Solutions)**

1. **The Odd-Even Rule (Asymmetric):** Give every friend a number. Odd numbers (1, 3, 5) must pick up their **left** chopstick first. Even numbers (2, 4)

must pick up their **right** chopstick first. This stops them from locking each other out.

2. **Limit the Chairs:** Only allow a maximum of **4 friends** at the table at the same time. This guarantees that at least one person will get both chopsticks, finish eating, and leave.
3. **All or Nothing:** A friend can only touch the chopsticks if **both** the left and right ones are completely free. If one is missing, they are not allowed to pick up either.

❖ Semaphore Solution

One simple solution is to represent every chopstick using a semaphore.

We can have:

Semaphore chopstick[5];

All chopstick semaphores are initially set to:

1

A philosopher uses:

wait()

to take a chopstick.

The philosopher uses:

signal()

to release a chopstick.

The basic code is:

while (TRUE)

{

wait(chopstick[i]);

wait(chopstick[(i+1) % 5]);

// eat

signal(chopstick[i]);

signal(chopstick[(i+1) % 5]);

// think

}